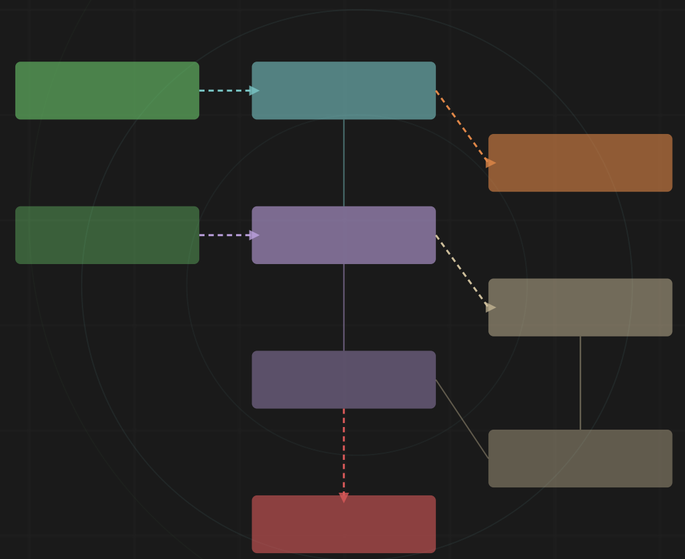
INTERNAL PRODUCTION REFERENCE
FOR ANIMATION SERIES — FROM ASSET TO FX & FINAL PIXEL

v1.0 · 2026

UNREAL. PIPELINE

FOR ANIMATION SERIES — FROM ASSET TO FX & FINAL PIXEL



This document covers **2 versions of the production workflow** — how we get from raw assets all the way to final FX & render inside Unreal.

Version 1 — Ideal. Full FBX Skeletal setup. Fast, efficient, full access to everything Unreal can do. Requires 100% Unreal-ready skeleton rig from the asset & rig department.

Version 2 — Less Ideal. Alembic-based. Still works, but production will slow down significantly the more characters you have.

01 Asset Creation → 02 Rigging → 03 Shading → 04 Animation → 05 Shot Setup → 06 FX & Postpro → 07 Lighting & Render

FBX SKELETAL | ALEMBIC | NIAGARA FX | LEVEL SEQUENCE | CONTROL RIG | LIGHTING & RENDERING

Internal Use Only · Do Not Distribute

UNREAL ENGINE 5
Production Reference & Requirements

Author · Adam RK

Version 1 - Ideal Pipeline
(Fastest Speed and Full benefit of unreal features)

Asset Creation

Modelling & Texture Creation
Outside Unreal (Maya/Blender & Substance/others). make sure we name all material & shading group inside the geo

Export the finished Geo&Textures (Without Rig) fbx/abc for Geo

png with : albedo,orm,normalmap

Preferably Asset & Rig department should prepare 2 version of asset.

Low Poly/proxy for animation & High Poly version for unreal.

Below unreal 5.5
Unreal Doesn't have subdivision. so we need to subdiv before setting up skeletal mesh for unreal

scroll down to end of page to see full rig requirement for unreal

Applying Texture &
Shader Creation Inside Unreal, looks good?

Rigging in Blender/Maya/Unreal
(between this 3, can choose which are we going to animate)

Export the finished Skeletal Part only on the rig (the Geo & Joints) into unreal to setup Skeleton fbx

Set/Big Stuffs import using "Import Into Scene"

to auto convert hierarchy of groups & meshes into Blueprint Scenes & Static Meshes

Setting Up Skeletal Mesh in Unreal with all the Shader, do deformation test.

Animation

Animate in Blender/Maya/Unreal

Export the Geo & Joints only into fbx. and import in unreal as a skeletal animation

Shot Setup in Unreal, All fbx cache imported here, build level sequence, check the cache

FX & Postpro

Complex FX Such as Heavy Sim can be done outside unreal, taking either anim file or the better cached animation

Simple FX Such as Particles
or Material FX Such as water ripple or grass&foliage sway can be directly done in Unreal

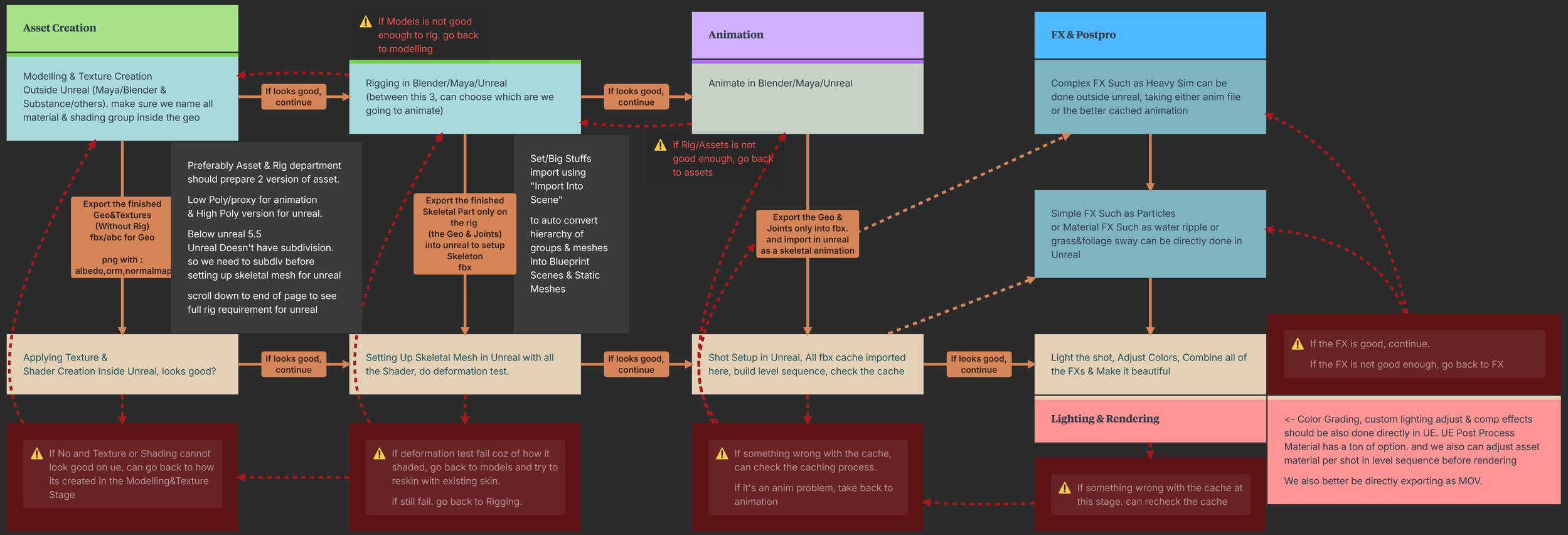
Light the shot, Adjust Colors, Combine all of the FXs & Make it beautiful

Lighting & Rendering

<- Color Grading, custom lighting adjust & comp effects should be also done directly in UE. UE Post Process Material has a ton of option. and we also can adjust asset material per shot in level sequence before rendering

We also better be directly exporting as MOV.

Version 1 - Ideal Pipeline
With flow of what goes where, and where it goes when something goes wrong



Version 2 - Less Ideal Pipeline - Using Alembic but we need to setup skeletal for any assets that are "skeletal-able"
(for projects that don't have the budget to have rigger to rig for 100% unreal requirement) - This will have a significant slow down with more numbers of assets

Creation or existing assets

New or exisiting assets that made outside of unreal, we need to do cleanup for atleast the Group Hierarchy, and most of the scale are uniform in the default state.

Export the finished Geo&Textures (Without Rig) abc for Geo

Applying Texture
& Shader Creation Inside Unreal. We make sure when we re-export&import abc again. unreal can auto pickup material trough the shading group names

Rigging in Blender/Maya
No special requirement except just for the grouping and naming.

Animation

Animate in Maya/Blender

Per shot, we will export into alembic cache per char.

then we will subdivide and cache again into a subdivided abc cache.

Shot Setup in Unreal, All Abc cache will be imported here and setup as a level sequence

FX & Postpro

Complex FX Such as Heavy Sim can be done outside unreal, taking either anim file or the better cached animation

Simple FX Such as Particles
or Material FX Such as water ripple or grass&foliage sway can be directly done in Unreal

If we don't have Unreal FX Artist available, Simple FX like this means doing it outside unreal and need compose it back into final pixel

Combine all of the fxs, Light the shot
& Make it beautiful

Lighting & Rendering.
if the render need more adjustments.
can render with layer & passes

⚠️ Then Compositing will happen here either using nuke/afterfx/fusion/etc.
again this is not ideal and means we already losing the benefit of realtime rendering

Version 2 - Less Ideal Pipeline - Using Alembic but we need to setup skeletal for any assets that are "skeletal-able" (for projects that don't have the budget to have rigger to rig for 100% unreal requirement) - This will have a significant slow down with more numbers of assets

Creation or existing assets

New or exisiting assets that made outside of unreal, we need to do cleanup for atleast the Group Hierarchy, and most of the scale are uniform in the default state.

Rigging in Blender/Maya
No special requirement except just for the grouping and naming.

⚠ If Model is not good enough to rig, go back to modelling

Export the finished Geo&Textures (Without Rig) abc for Geo

Since we are using ABC.
we just need to make sure if we can cleanly subdivide it and the animation will work while being subdivided. coz we are going to cache the geo using alembic

The Shading Group must have a clear name. coz abc don't bring materials, abc only bring shading group. and in UE, we will reassign using SG name.

Applying Texture
& Shader Creation Inside Unreal. We make sure when we re-export&import abc again. unreal can auto pickup material trough the shading group names

⚠ If Material cannot be auto pickup by unreal using the shading group names, can go back to

If looks good, continue

Animation

Animate in Maya/Blender

⚠ If Rig/Assets is not good enough, go back to assets

Per shot, we will export into alembic cache per char.

then we will subdivide and cache again into a subdivided abc cache.

Shot Setup in Unreal, All Abc cache will be imported here and setup as a level sequence

⚠ If cache is error, can go back to cache process, if cache is as animation then go back to animation

If all good but something in the shader was wrong, go back to Shader Creation Stage

If looks good, continue

FX & Postpro

Complex FX Such as Heavy Sim can be done outside unreal, taking either anim file or the better cached animation

Simple FX Such as Particles
or Material FX Such as water ripple or grass&foliage sway can be directly done in Unreal

If we don't have Unreal FX Artist available, Simple FX like this means doing it outside unreal and need compose it back into final pixel

⚠ If FX Is not good enough when got composed into final pixel, get back to FX

Combine all of the fxs, Light the shot
& Make it beautiful

Lighting & Rendering.
if the render need more adjustments.
can render with layer & passes

⚠ Then Compositing will happen here either using nuke/afterfx/fusion/etc.
again this is not ideal and means we already losing the benefit of realtime rendering

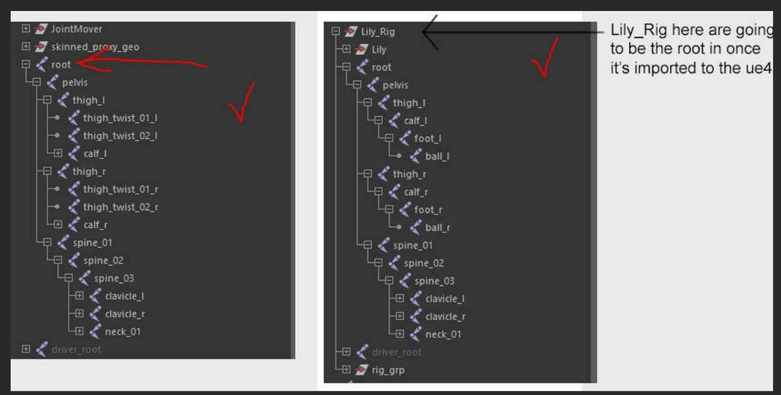
⚠ If something wrong with the cache at this stage. can recheck the cache

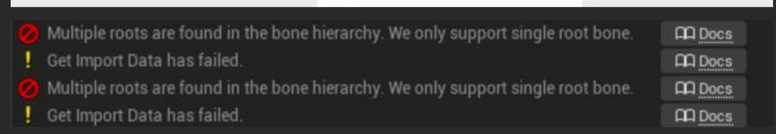
⚠️ Detailed Requirements and more Important Info

Version 1 is my recommended pipeline and the fastest way to work. By going Full Skeletal FBX, we're working natively inside Unreal, which means we get real-time performance, can do direct control rigs, and seamless FX integration, without the overhead of managing external geometry caches every time something changes.

https://docs.unrealengine.com/en-US/Engine/Content/Importing/FBX/SkeletalMeshes/ this is basically the standard way of doing it in games. This way, unreal currently only supports only Skinned Geometry to Joints & Single layer Blendshapes.

Couple of notes how the hierarchy inside the rig is best for unreal are:

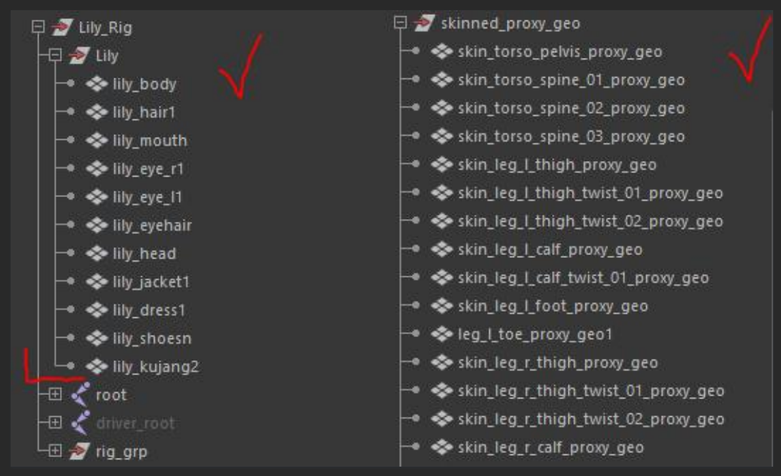




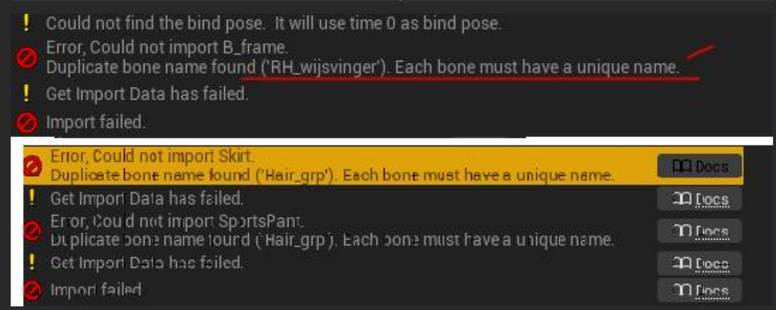
a. Joints are single rooted. For example below, every ***"joints that skinned to the geo"*** are in one hierarchy, the left one are from the Basic_Biped sample (the unreal default mannequin), which shows, even the root are in the root of the scene.

However on the right are a rig made using ARTv2 (animation & rigging toolkit, originally artv1 provided by epic games to rig characters their way). Show that the *"joints that skinned to the geo"* can be grouped along the mesh group, and other joints that don't skinned to geo, like driver joints can be stored somewhere else, and didn't get selected and exported when exporting fbx animation.

These hierarchies are for avoiding errors, like these. Reason that unreal only support single root is originally for how unreal handles character in games, root is accounted for moving the character capsule, collisions, and where the character position in the world.

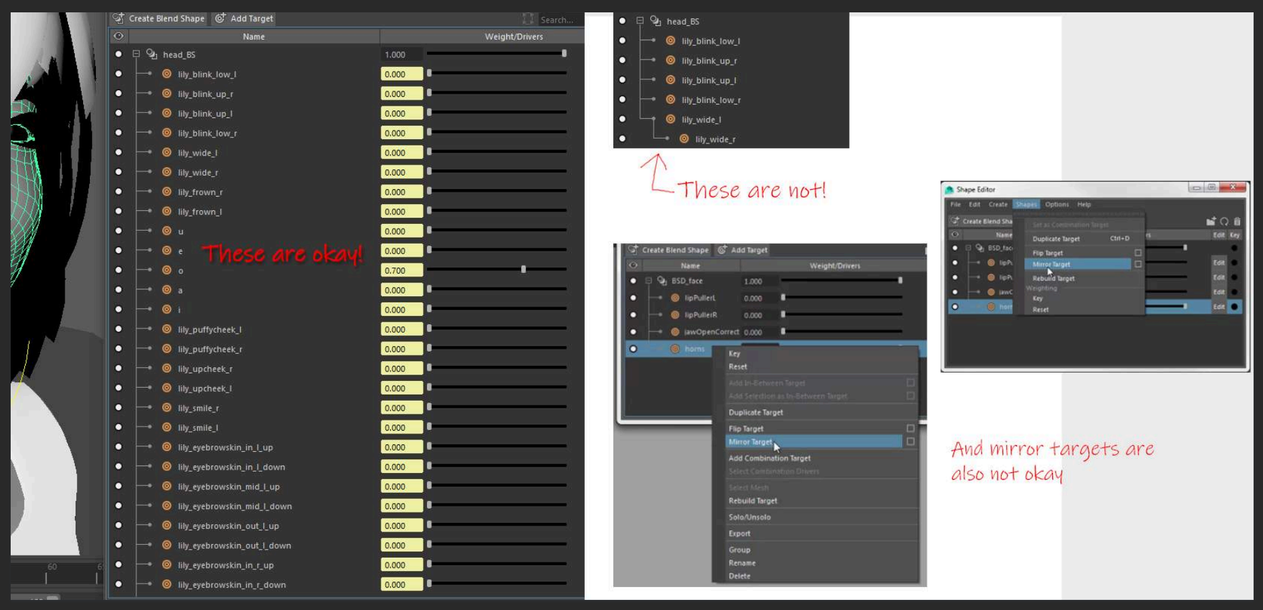


b. Best to put meshes in single group without any group inside the group. Taken from unreal example (Basic_Biped), all the meshes are in single group hierarchy. To note, having some grp inside the mesh group are mostly okay and rarely have a problem.



c. No duplicate name / multiple entities having the same name. Unreal will spit out error like these:

reason are unreal use specific strings taken from the name of the bone, like physics asset that generated from the joints name, collisions, socket attachment etc.



d. No blendshapes that being blendshaped. Unreal currently only supports single layer blendshape. Mesh that had blendshape are supported as long as the blendshape are not being blendshape by another blendshape.

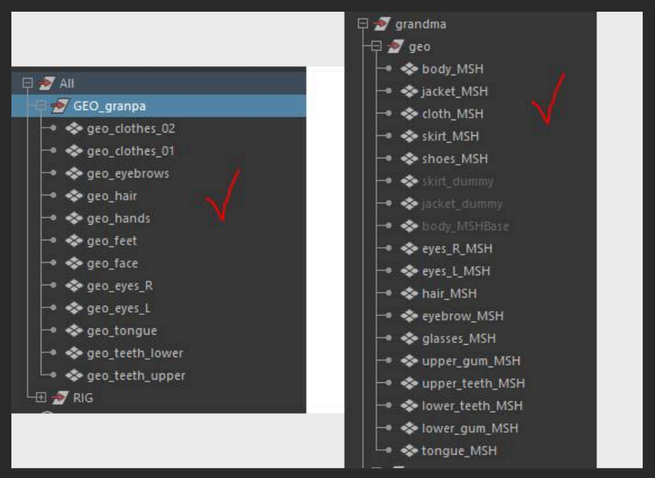
Also on that note, mirror targets on blendshapes are also not working once it imported to unreal. As Maya doesn't actually "have" the mirrored shape, it only mirrors the calculation inside Maya.

Version 2 Alembic-based. Still works, but every time animation changes, we need to re-export, re-subdivide, and re-import the cache again — per character, per shot. So the more characters you have, the more that pain multiplies. Alembic are heavy, it's heavy to subdivide, heavy to bake, heavy to cache, heavy to import & heavy to run.

https://docs.unrealengine.com/en-US/Engine/Content/Importing/AlembicImporter/#importasgeometrycache

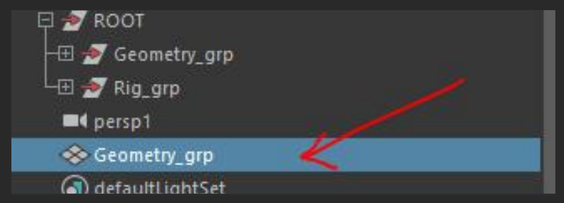
Alembic is a lot more forgiving in the setup/asset creation stage since, unreal basically support most of the geometry that alembic can cached.
Although some problems are emerged when exporting alembic without preparation. Like crashes and geometry that walks in place once it's exported.

Best practices are the hierarchy again are need to be really well structured, and I recommend having all the meshes in a single group and no group inside it, similar how the mesh group structured in skeletal mesh before.



When exporting from a bunch of mesh with different groups cause a lots of issue like crashes in unreal 4. at the time I was testing it with unreal 4.26.2.

But as we tested yesterday. It Still very safe to select the whole geo group(even there is more groups inside), combine the mesh.



Other things to notes are unlike dcc apps like Maya, unreal has no subdivision. Since unreal is a game engine, everything is triangulated. So subdivision like "opensubdiv" are not anywhere near being worked on unreal.

Even with Nanite Skeletal Mesh, Unreal wont add more geo/mesh resolution. Nanite only allow highpoly mesh to be optimized(reduced at distance) at runtime.

- Last one is Visibility.
  Visibility is still a problem since Alembic in UE4, in UE4 basically visibility key/data is ignored by unreal. UE5 can read abc visibility data, but only if the mesh is not being combined. but when the mesh is not combined, the imported track will be separated for each mesh, and if we choose to flatten tracks in unreal, the FaceSet is gone. (the data that brings which face has which shading groups).

  so to work around that, we just not use any visibility key. instead we zero out the scale and move them underground for everymesh that's been toggled off from controller.